

Tutorial: Statistical N-gram Language Model

Presenter: Hanxiao Jiang (Shawn)

Reference: <https://medium.com/codex/statistical-language-model-n-gram-to-calculate-the-probability-of-word-sequence-using-python-2e54a1084250>

Overview

1. Type of language models
2. N-gram definition
3. N-gram language model
4. The process to compute the n-gram LM from corpus
5. How to implement

Language Model

- Language Model (LM): predict the probability of the next word
 - Statistical Language Model
 - N-gram Language Modelling (Our focus)
 - Neural Language Model
 - Use neural network to predict the probability

What is N-gram?

- N-gram: Unigram, Bi-gram, Tri-gram N-gram
- A simple sentence: I don't like doing homework
 - Unigram: {"I", "don't", "like", "doing", "homework"}
 - Bigram: {"I don't", "don't like", "like doing", "doing homework"}
 -

N-gram Language Model?

- How to use above definitions to construct a language model?
 - Probability of the next words (w_n)
 - $P(w_n) \Rightarrow$ Unigram probability
 - $P(w_n|w_{n-1}) \Rightarrow$ Bigram probability
 -

- For unigram:

$$P(w_i \mid w_1 w_2, \dots w_{i-1}) \approx P(w_i)$$

- For bigram:

$$P(w_i \mid w_1 w_2, \dots w_{i-1}) \approx P(w_i \mid w_{i-1})$$

How to get the probability from the corpus

- Use frequency of the words in the corpus to calculate the probability
- The simplest one: unigram $P(w) = \text{freq}(w)$
- Bigram:
 - Need to count the frequency of both unigram and bigram

$$P(w_n \mid w_{n-1}) = P(w_n) / P(w_n, w_{n-1})$$

The process to compute the n-grams LM from corpus

1. Determine the n value in n-gram

- Some tradeoffs:
 - Shorter n-grams will have less sparsity issue since there are fewer combinations, so the estimated probabilities are more accurate
 - Longer n-grams can consider more past words, and so can capture important information from before

2. Calculate the frequency of unigram and n-gram

- a. They are treated as joint probability $\Rightarrow$ used to calculate the conditional probability

3. Process the whole corpus

How to use the n-gram probabilities

- Calculate the probability of a sentence

$$P(w_{1:n}) = \prod_{k=1}^n P(w_k | w_{1:k-1})$$

Example (Bi-gram): add “eos” before the first word and after the last word

$P(\text{“I do not like green eggs and ham”})$

$= P(I | \text{eos}) * P(\text{do} | I) * P(\text{not} | \text{do}) * P(\text{like} | \text{not}) * P(\text{green} | \text{like}) * P(\text{eggs} | \text{green})$

$* P(\text{and} | \text{eggs}) * P(\text{ham} | \text{and}) * P(\text{eos} | \text{ham})$

How to implement (1)

- You need some corpus for your language model
 - Toy example with only a few words
 - Here are only five simple sentences
 - This is the simplest case
 - There are no punctuation marks
 - All characters are lower-case

```
1 corpus = ["i love you", "i love him", "he love you", "he love me", "you love me"]
```

```
1 corpus = ["i love you", "i love him", "he love you", "he love me", "you love me"]
```

How to implement (2)

- How to get the unigram probability

```
3 unigrams = {}
4 total_unigrams = 0
5 for sentence in corpus:
6     words = sentence.split(" ")
7     total_unigrams += len(words)
8     for word in words:
9         if word not in unigrams:
10             unigrams[word] = 1
11         else:
12             unigrams[word] += 1
13
14 for k, v in unigrams.items():
15     unigrams[k] /= total_unigrams
16
17 print(unigrams)
```

```
• (base) Downloads/NLP_turtorial [ python n_gram.py ] 8:09 PM
{'i': 0.1333333333333333, 'love': 0.3333333333333333, 'you': 0.2, 'him': 0.06666666666666667, 'he': 0.1333333333333333, 'me': 0.1333333333333333}
```

```
1 corpus = ["i love you", "i love him", "he love you", "he love me", "you love me"]
```

How to implement (3)

- How to get the bigram probability

```
19 bigrams = {}
20 total_bigrams = 0
21 for sentence in corpus:
22     words = sentence.split(" ")
23     # We want to consider the EOP
24     total_bigrams += len(words) + 1
25     for i in range(len(words)+1):
26         if i == 0:
27             bigram = (None, words[i])
28         elif i == len(words):
29             bigram = (words[i-1], None)
30         else:
31             bigram = (words[i-1], words[i])
32         if bigram not in bigrams:
33             bigrams[bigram] = 1
34         else:
35             bigrams[bigram] += 1
36
37 for k, v in bigrams.items():
38     bigrams[k] /= total_bigrams
39
40 print(bigrams)
```

```
(base) Downloads/NLP_tutorial [ python n_gram.py ] 8:53 PM
{(None, 'i'): 0.1, ('i', 'love'): 0.1, ('love', 'you'): 0.1, ('you', None): 0.1, ('love', 'him'): 0.05, ('him', None): 0.05, (None, 'he'): 0.1, ('he', 'love'): 0.1, ('love', 'me'): 0.1, ('me', None): 0.1, (None, 'you'): 0.05, ('you', 'love'): 0.05}
```

Calculate the probability of sentences

- Unigram

- “you love him” = 0.00444444444444444444
- “you love me” = 0.00888888888888888889
- You love me more? Nope, corpus is not big enough

```
42 sent = "you love him"
43 prob = 1
44 words = sent.split(" ")
45 ✓ for word in words:
46     | prob *= unigrams[word]
47
48 print(prob)
```

Calculate the probability of sentences

- Bigram

- “you love him” = $6.2500000000000002 \times 10^{-6}$
- “you love me” = $2.5000000000000008 \times 10^{-5}$
- You love me more? Nope, corpus is not big enough.

```
sent = "you love him"
prob = 1
words = sent.split(" ")
for i in range(len(words) + 1):
    if i == 0:
        prob *= bigrams[(None, words[i])]
    elif i == len(words):
        prob *= bigrams[(words[i-1], None)]
    else:
        prob *= bigrams[(words[i-1], words[i])]

print(prob)
```

Potential problems?

- If a word does not appear in the corpus, we cannot handle it
 - Add some virtual probability for other words
 - A large enough corpus can help to alleviate this problem
 - You will learn more in the NLP main course
- Bigrams are limited in the length of the statistical dependencies they can handle
 - “I love me” can also have a relative high probability
 - But trigram can deal with such problem
 - However, we need larger corpus to prepare good probability estimates for trigrams

Additional Resources

1. Our lecture 2: language modelling
2. Using character-level n-grams to generate Shakespeare and code

https://nbviewer.org/github/anoopsarkar/nlp-class/blob/gh-pages/assets/notebooks/goldberg_lm.ipynb

Thank you